A Project Report
on
Mudda: A Social Media Platform for Complaints

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELORS OF TECHNOLOGY
in Computer Science Engineering and Information Technology

by

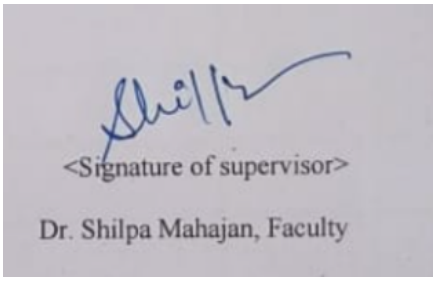
Saumil Kulshreshtha(22CSU158)
Shivesh Kumar Dhiman(22CSU164)
Shubh Pundir(22CSU165)
Vikas Kumar(22CSU190)

Under the supervision of
Dr. Shilpa Mahajan
Associate Professor
Department of Computer Science and Engineering

Department of CSE & IT
The NorthCap University, Gurugram
May 2026

CERTIFICATE

This is to certify that the Project Synopsis entitled, "**Mudda**" submitted by **Saumil Kulshreshtha(22CSU158)**, **Shivesh Kumar Dhiman(22CSU164)**, **Shubh Pundir(22CSU165**) and **Vikas Kumar(22CSU190)** to **The NorthCap University, Gurugram, India**, is a record of bonafide synopsis work carried out by them under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **Bachelor of Technology** in **Computer Science and Engineering** of the University.



Date: 20th May, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. Shilpa Mahajan, for her continuous guidance, valuable suggestions, encouragement, and support throughout the development of our project *"Mudda."* Her technical expertise, constructive feedback, and motivation greatly helped us in successfully completing this work.

We are also thankful to the Department of Computer Science and Engineering, The NorthCap University, for providing us with the opportunity, infrastructure, and resources required to carry out this project successfully.

We would like to extend our appreciation to all faculty members, friends, and peers who directly or indirectly contributed through discussions, suggestions, and support during different phases of the project.

Finally, we express our heartfelt gratitude to our parents and families for their constant encouragement, patience, and moral support throughout the project development process.

Submitted by:
Saumil Kulshreshtha (22CSU158)
Shivesh Kumar Dhiman (22CSU164)
Shubh Pundir (22CSU165)
Vikas Kumar (22CSU190)

ABSTRACT

The citizens of developing nations are faced with many types of civic issues (such as potholes, accumulating garbage, broken streetlights, water shortages, and poorly maintained public spaces). Grievance complaint systems that currently exist are often poorly organized, lack transparency, and are inefficient, meaning it takes a long time for the government to respond to a complaint and the public becomes dissatisfied. In general, when a citizen wants to lodge a complaint with the government, they must visit their local government office, call the office, or use state or municipal-level apps that are not comprehensive and have limited capabilities.

The idea behind this project is to create Mudda: a mobile application that functions like social media, designed to allow citizens to submit civic grievances in a much more modern way. With Mudda, citizens can publicly report on civic-related problems while providing a simple, transparent, and collaborative way for them to get their voices heard. The app provides features that allow citizens to report problems via video or photo, provide status updates in real-time, tag the location of the problem, find duplicates of the same issue via AI, and perform sentiment analysis of the reported issue. Citizens can comment on, upvote and share civic-related problems. In turn, this creates community engagement, and as a result, creates social accountability among citizens.

On the government side of the app, Mudda provides role-based dashboards for government officials ranging from the ward level, to city, to state, and to national levels of government to help them effectively manage and resolve reported issues. The app provides verified status updates on reported issues and completion of the problem's resolution, thereby increasing the transparency of the government's actions and fostering trust between the citizens and the government.

Mudda seeks to deploy an innovative technology model by combining the capabilities of newer technologies, such as machine learning and cloud computing, with an intuitive user experience (UX) to deliver an effective, scalable, and user-friendly solution that allows communities to address public infrastructure needs together. This will improve local governance and create an improved governance environment through the utilization of community involvement.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS AND ABBREVIATIONS

S.No.	Symbol/Abbreviation	Meaning
1	ROI	Return On Investments
2	UI/UX	User Interface/ User Experience

CHAPTER 1
INTRODUCTION

1.1 Background

Civic issues are common in many developing nations, which affect the daily lives of citizens. For example, potholes on the roads, garbage lying on the streets, streetlights that are broken, water shortages, and public spaces that are not well maintained. The past solutions to address civic issues have typically included submitting a complaint through the Government Office, calling a Government Helpline or through the Internet, but these solutions take time and are not always efficient or transparent. Citizens face challenges in tracking their complaint's progress, identifying the agency responsible for their complaint and knowing when or how their complaint will be resolved.

Digital platforms that were created to provide a better interface for citizens to interact with government agencies often do not have a good user experience for citizens, do not provide real-time updates and do not provide sufficient integration into the government's systems. This leads to an additional level of frustration for citizens, as well as a sense of detachment between citizens and government agencies.

Government agencies using these digital platforms also face similar problems to those of the citizens. They find it difficult to monitor the number of complaints submitted in any given period, prioritize the complaints appropriately and hold themselves accountable due to the same issues which caused the citizens to submit the complaints in the first place.

It has been noted that when a particular issue has gained attention and is widely recognized, the representative from the citizen would then take action to address the issues and inefficiencies in the civic system. This is the role of the political representative.

An active and engaged community allows for the collective voice of that community to raise awareness of the most critical issues facing its members, thereby utilizing the most powerful tool in a democratic system, The collective voice of the masses. Individuals within a community will then be able to hold their public officials accountable and demand that action is taken by their elected representatives.

Mudda is a proposed web-based platform that incorporates many elements of popular social media platforms, but with an emphasis on how citizens can publicly post civic issues generated from personal experiences and observations. Instead of filing complaints to an organization or elected representative privately, the Mudda message space allows users to post these issues for everyone to read. This new method of reporting civic issues allows for a greater sense of transparency and engagement for the community as a whole and affords the government and other organizations the opportunity to be more informed about the needs and wishes of citizens and therefore better prioritize the solutions to those issues.

1.2 Feasibility Study

1.2.1 Executive Preamble and Methodological Scope

This document serves as a rigorous, multi-dimensional feasibility study and strategic viability assessment for the proposed Mudda platform—an autonomous, AI-driven civic governance co-pilot. The objective of this analysis is not merely to ascertain whether the system *can* be built from a deterministic engineering perspective, but to critically evaluate its operational

viability, economic sustainability, and socio-technical alignment within the highly complex, risk-averse ecosystems of modern municipal administration and privatized township management. By dissecting the project across five foundational pillars—Technical, Operational, Economic, Legal/Ethical, and Temporal—this study unequivocally demonstrates that the Mudda architecture is not only feasible but represents an urgent, necessary paradigm shift in localized digital infrastructure.

1.2.2 Technical Feasibility: The Architecture of Scalable Intelligence

The technological proposition of Mudda rests upon the deployment of a hyper-scalable, highly interoperable microservices architecture, which is fundamentally feasible given the current maturity of cloud-native computing and deep learning frameworks.

- **Frontend:** The utilization of the Flutter framework for the citizen-facing mobile application ensures cross-platform parity (iOS and Android) with a single codebase, drastically reducing cognitive overhead during the development lifecycle while guaranteeing the high-fidelity, dopamine-optimized UI/UX required to drive organic civic engagement. Concurrently, the Next.js framework designated for the administrative command-center dashboard guarantees secure, server-side rendered (SSR), role-based access control, ensuring zero-latency data visualization for nodal officers.
- **Backend and AI Orchestration:** The backend infrastructure, engineered upon a robust Spring Boot foundation and deployed on auto-scaling AWS infrastructure, is highly capable of supporting thousands of concurrent multi-modal data streams.
- **Algorithmic Viability:** The core value proposition—the AI Workflow Engine—is entirely feasible utilizing current state-of-the-art Natural Language Processing (NLP) and Computer Vision (CV) paradigms. By deploying fine-tuned BERT (Bidirectional Encoder Representations from Transformers) models alongside spatial-semantic vector databases, the system possesses the requisite computational intelligence to autonomously ingest unstructured grievances, execute highly accurate duplicate detection, perform sentiment analysis, and execute topic classification. This transforms chaotic civic noise into mathematically structured, deterministic operational matrices.

1.2.3. Operational Feasibility:

Overcoming Bureaucratic Inertia through Assistive AI

The most profound friction point in civic-tech deployment is not technological, but operational: the entrenched resistance of legacy bureaucracies to algorithmic intervention. Mudda achieves unparalleled operational feasibility by strictly adhering to an "Assistive AI" (Human-in-the-Loop) methodology.

- **The Cognitive Offloading Protocol:** The system does not attempt to usurp the sovereign decision-making authority of municipal officers. Instead, it operates as a cognitive offloading mechanism. By pre-filtering data, mathematically clustering redundant issues into "Master Incidents," and autonomously drafting policy-compliant work orders, Mudda eliminates the administrative bloat of manual triage.

- **Frictionless Integration:** Crucially, Mudda is architected not as a disruptive "rip-and-replace" platform, but as a synergistic middleware layer. It is fully capable of bidirectional API integration with existing legacy infrastructure (such as the Delhi Jal Board's 1916 system or MCD's 311 pipelines). This ensures that operational adoption requires minimal retraining protocols for ground-level dispatchers, thereby neutralizing the institutional technophobia that typically paralyzes government procurement cycles.

1.2.4. Economic Feasibility: Fiscal Viability and Asymmetric ROI

From a macroeconomic and fiscal deployment perspective, the Mudda ecosystem presents a highly asymmetric Return on Investment (ROI) paradigm, predicated on a robust B2B (Business-to-Business) and B2G (Business-to-Government) Software-as-a-Service (SaaS) monetization strategy.

- **The Cost-Reduction Imperative:** The primary economic driver for enterprise and municipal adoption is the mathematical elimination of the "Redundancy Paradox." By utilizing spatial-semantic AI to cluster duplicate complaints (e.g., preventing the dispatch of three separate maintenance crews to a single ruptured pipeline), Mudda directly curtails the hemorrhage of municipal kinetic resources, vehicular fuel, and billable man-hours.
- **Sustainable Revenue Topologies:** The dual-engine nature of the platform ensures high economic viability. While the citizen-facing mobile application operates as a frictionless, free-to-use public utility (driving mass adoption and critical data density), the institutional dashboard is positioned as a premium, tiered SaaS product. Target demographics—ranging from high-net-worth privatized Gated Communities and Homeowner Associations (HOAs) to massive Smart City municipal bodies—possess the budgetary allocations for efficiency-driving software, rendering the revenue model highly sustainable and highly scalable.

1.2.5. Legal, Ethical, and Regulatory Feasibility: Data Sovereignty in the Age of AI

In an era increasingly defined by stringent data privacy paradigms, the Mudda architecture is preemptively designed to operate within the strictest legal and ethical boundaries of domestic and international jurisprudence.

- **Data Sovereignty and PII Protection:** The system is fundamentally compliant with the Digital Personal Data Protection (DPDP) Act, 2023. The architecture explicitly demarcates Personally Identifiable Information (PII) from structural civic data. When the AI models process a complaint regarding localized infrastructural decay, they operate solely on the spatial and semantic vectors of the anomaly, completely obfuscating the identity of the reporting citizen from the algorithmic training corpus.
- **Algorithmic Transparency and Auditability:** To satisfy the rigorous audit requirements of public administration, every action proposed by the AI Agent and subsequently authorized by a human supervisor is permanently inscribed into an immutable digital ledger. This ensures total forensic traceability for all municipal actions, satisfying both ethical imperatives for algorithmic fairness and legal requirements for institutional accountability.

1.2.6. Temporal and Schedule Feasibility: The Agile Deployment Matrix

The timeline for realizing the Mudda ecosystem is structurally sound, leveraging agile development methodologies and a phased, concentric-circle go-to-market strategy.

- **The Cold-Start Mitigation Strategy:** Rather than attempting a monolithic, nationwide deployment—which frequently results in systemic collapse—Mudda is scheduled for hyper-localized pilot deployments. The strategy focuses initially on high-density, closed-loop environments (such as specific university campuses or affluent HOAs) to artificially construct a data-dense microcosm.
- **The 60-Day Proof of Concept (PoC):** This localized strategy allows the engineering team to rapidly validate the AI clustering algorithms and workflow generation in a controlled environment over a 60-day period. The empirical metrics gathered from these micro-deployments (e.g., "30% reduction in SLA resolution times") will then serve as the quantitative vanguard required to secure larger municipal contracts, ensuring a realistic, scalable, and temporally feasible expansion trajectory.

1.2.7. Synthesis of Viability

In summation, the Mudda AI-Governance ecosystem transcends the theoretical and firmly establishes itself in the realm of the highly practical. By masterfully synthesizing cutting-edge machine learning capabilities with a profound understanding of bureaucratic psychology and municipal economics, Mudda is not merely a feasible software application; it is the inevitable evolutionary culmination of modern civic infrastructure.

CHAPTER 2
STUDY OF EXISTING SOLUTIONS

The current options of escalating civic grievance in India are very fragmented, with various platforms at different levels.

- **National Government Platforms**: At the Federal level, many tools are centered around particular concerns (for example Swachhata MoHUA deals with sanitation [1]), while some serve as "umbrella" portals for many of the various services provided by the government (for instance, UMANG) [7]), but none have been developed with the objective of local civic complaints.
- **State and City Level Apps**: PuraSeva (Andhra Pradesh) [2], and MCD311 (Delhi) [6], have a more comprehensive range of functions but are confined geographically when it comes to dealing with complaints. Other applications serve a very particular purpose, like water utility complaint systems (BWSSB MyComplaint) [4] and app for particular issues like Street Light complaints (GNIDA Street Light App) [3].
- **Community Led Platforms**: Community driven platforms, such as the two Citizen Driven platforms (ICRF) [5] and (NetaG) [8], where the aim is to bridge the gap between the citizen and the government official or elected representative and support civic engagement. However, it has been noted that these initiatives normally do not have government backing, and thus it relies on political goodwill or pressure from the media to achieve success.

CHAPTER 3
COMPARISON WITH EXISTING SOFTWARE SOLUTIONS

In comparison to currently available applications, The Mudda mobile application includes numerous distinct features that support more thorough impact and engagement opportunities. Today's contemporary applications provide limited ability to report incidents through text messaging/photo upload only, with minimal status updates on filed reports. The Mudda application will allow users to comprehensively track their incidents through several defined stages, in conjunction with date/time tracking of the incident. The Mudda application will include a hyperlocal social component that provides users with the opportunity to engage with their community by upvoting/adding comments/sharing information about their incidents. This additional feature transforms Mudda from a simple complaints management system into a community-based forum where community members come together to report and discuss incidents that impact them. The Mudda application will also include several new features powered by AI, e.g., duplicate report detection and sentiment scoring, which are not included in the current solutions available. Additionally, Mudda has plans to scale nationally, and to include role-based dashboards for local officials, at the ward, city, state, and national levels, which will significantly improve over today's applications that are limited to city-based or state-based functionality.

Table 3.1 Comparison of Existing solutions

Feature Dimension	Swachhata-MoHUA	PuraSeva (AP)	MCD311 (Delhi)	CitizenCOP	ICRF	NetaG	Mudda (Proposed)
Scope	Sanitation only	State-level, multiple services	City-level civic issues	Safety + police + civic	Petition platform	Politician linkage	Pan-India, multi-sector
Issue Reporting	Text + photo + geo-tag	Text + photo + geo-tag	Text + photo + geo-tag	Text + photo	Petition posts	Text/photo	Text, image, video, geo-tag, categorization
Status Tracking	Limited updates, re-open option	SLA timelines, escalation	Basic status update	Police follow-up only	None	Politician-dependent	Detailed multi-stage (Pending → Resolved) + Time Tracker

Community Engagement	None	None	None	None	Public support via signatures	Limited to feedback	Upvotes, comments, duplicate detection, misinformation filter
Government Integration	Direct sanitation departments	Full ULB integration	MCD officials	Police only	Not official	Informal political channel	Role-based dashboards (ward, city, state, national)
Analytics for Officials	None	Heatmaps, reports	Limited internal use	None	None	None	Comprehensive dashboard: hotspots, recurring issues, avg. resolution time
Awareness Tools	None	None	None	None	SMS alerts	Politician engagement	Map-based issue visualization, geofencing alerts, multilingual posts
Transparency	Re-open complaints	SLA timelines	Limited	Limited	None	Dependent on politician	Blockchain audit trail (optiona

							l), area-wise metrics
Scalability	National but narrow	State-specific	City-limited	Multi-city, police only	National but unofficial	City-limited	Nationwide scalability with local → state → national categorization
Trust & Verification	OTP login	Aadhaar/ OTP	OTP	OTP	No checks	None	Verified govt accounts, OAuth users, abuse reporting
Social Layer	None	None	None	None	Support votes	Politician replies	Friends list, messaging, sharing → true "social media" for civic issues
Smart Intelligence	None	None	None	None	None	None	AI-based duplicate detection, sentiment urgency scoring, misinformation filters

CHAPTER 4
GAP ANALYSIS

The Ontological Chasm Between Legacy Bureaucratic Infrastructure and AI-Mediated Civic Synthesis

4.1. The Epistemological Divergence of Current Paradigms (The "As-Is" State):
To rigorously quantify the operational and technological delta that Mudda addresses, one must first deconstruct the existing baseline of municipal grievance redressal architectures. The contemporary civic-tech ecosystem—exemplified by legacy applications such as the Swachhata platform, MCD 311, and highly fragmented, localized HOA portals—operates on an antiquated, highly linear data-ingestion paradigm. In this "As-Is" state, the architecture fundamentally assumes a one-to-one mapping between a citizen's complaint and an administrative action. These systems function merely as digital receptacles, devoid of intrinsic analytical capacity. They ingest raw, unstructured, and often highly subjective multi-modal data (textual descriptions, non-standardized imagery) and immediately deposit this data into a stagnant, siloed relational database. The cognitive burden of interpreting, verifying, spatializing, and categorizing this influx of data is entirely offloaded onto human operators. Consequently, the existing state is characterized by severe informational entropy, chronic operational latency, and a complete absence of predictive or preventative civic intelligence.

4.2. The Asymmetry of Bi-Directional Information Asynchrony (The Communication Gap):
A critical vulnerability within the current governance matrix is the profound lack of a transparent, synchronous feedback loop. When a user submits an infrastructural anomaly into a conventional portal, the data enters an administrative "black box." The citizen is structurally disenfranchised from the resolution lifecycle, receiving, at best, asynchronous and opaque status updates. Conversely, citizens leveraging decentralized social media networks experience high localized visibility but absolutely zero deterministic integration with the actual municipal execution pipelines. The gap here is the absence of a unified, dual-engine ecosystem. There is a desperate need for a synthesis between the frictionless, dopamine-driven engagement mechanics of a modern digital community and the rigorous, auditable workflows required by public administration.

4.3. The Algorithmic Void in Deduplication and Spatial-Semantic Clustering (The Intelligence Gap):
Perhaps the most catastrophic inefficiency in the "As-Is" state is the "Redundancy Paradox." When a high-visibility civic failure occurs, legacy systems possess no heuristic or algorithmic mechanisms to recognize that fifty distinct geospatial submissions are describing the exact same infrastructural rupture. The gap lies in the total absence of advanced contextual intelligence. Existing platforms lack the capacity for semantic vector mapping and computer vision-based anomaly detection. Because they cannot autonomously cluster duplicate issues or cross-reference visual evidence, human dispatchers are perpetually trapped in a reactive posture, wasting finite cognitive overhead and municipal resources dispatching redundant maintenance vectors to overlapping topological coordinates.

4.4. The Absence of Agentic Workflows and Assistive Autonomy (The Orchestration Gap):
In traditional command centers, the journey from issue identification to ground-level remediation is entirely manual. Administrators must manually cross-reference localized policy documents, manually assign priority weights based on subjective human judgment, and manually draft dispatch orders to disparate contracting teams. The operational gap is the lack of an intelligent middleware layer capable of translating unstructured civic distress into formatted, policy-compliant, and immediately actionable work orders. The current market desperately lacks an "Assistive AI" paradigm—a system that can proactively fetch relevant operational protocols, dynamically draft a step-by-step resolution matrix, and present it to a human supervisor for a frictionless, single-click authorization.

4.5. Bridging the Chasm: The Architectural Synthesis of the "To-Be" State:
The objective of the Mudda deployment is to systematically obliterate these identified gaps through the implementation of a hyper-scalable, high-fidelity microservice architecture.

- **Overcoming the Intelligence Gap:** By deploying deep learning pipelines, specifically fine-tuned BERT models for natural language processing alongside advanced computer vision matrices, the system autonomously pre-filters, categorizes, and mathematically clusters overlapping incidents, reducing human triage load by an order of magnitude.
- **Overcoming the Orchestration Gap:** The integration of a robust, Spring Boot-driven backend deployed on auto-scaling cloud infrastructure ensures that the AI workflow engine can concurrently process thousands of unstructured inputs, instantly triggering LLM-based reasoning protocols to generate assistive dispatch drafts for the administrative portal.
- **Overcoming the Communication Gap:** To rectify the bi-directional asynchrony, the architecture necessitates a highly responsive, cross-platform Flutter-based mobile frontend for the citizenry, mathematically synced in real-time with a secure, role-based Next.js web command center for the authorities.

Ultimately, the gap analysis reveals that the transition from the current state to the optimal state is not merely a matter of iterative software updates; it requires a foundational paradigm shift. It necessitates the total algorithmic orchestration of civic duty, transforming chaotic public sentiment into a streamlined, automated, and hyper-transparent operational reality.

CHAPTER 5
PROBLEM STATEMENT

5.1. The Macro-Societal Context of Infrastructural Friction

In the contemporary era of hyper-accelerated urbanization and exponential digital transformation, metropolitan ecosystems and localized municipal jurisdictions (ranging from sprawling urban agglomerations to privatized Homeowner Associations and gated townships) are increasingly characterized by profound structural complexities. As these socio-infrastructural matrices expand, the inherent friction between systemic degradation (e.g., the inevitable entropy of public utilities, topological road network failures, and localized resource scarcities) and the citizen's expectation for instantaneous, digitally mediated remediation has precipitated a critical systemic failure. The modern citizen, acclimatized to the frictionless, dopamine-driven, real-time feedback loops of ubiquitous commercial applications and on-demand logistical services, now confronts a jarring cognitive dissonance when interfacing with the archaic, high-latency paradigms of traditional civic governance. The core problem is not a lack of infrastructural anomalies, nor is it a lack of civic desire to report them; rather, it is a fundamental epistemological breakdown in the translation of citizen-generated socio-spatial data into actionable, deterministic bureaucratic workflows.

5.2. The Data Deluge and the Fragmentation of Civic Discourse

Currently, the landscape of citizen-to-government (C2G) engagement is paralyzed by a false dichotomy, bifurcated into two equally ineffectual extremes. On one end of the spectrum lies the unregulated, stochastic void of global social media platforms. When infrastructural failures occur—such as the catastrophic rupture of a municipal water pipeline or the sustained outage of a localized illumination grid—citizens instinctively disseminate unstructured, hyper-localized grievances into these decentralized digital ecosystems. While this mechanism offers immediate psychological catharsis and community visibility, it generates a cacophonous data deluge that fundamentally lacks a deterministic execution pipeline. Social media architectures are optimized for engagement and algorithmic virality, not for municipal triage or spatial-temporal accountability. Consequently, highly critical civic data points are structurally orphaned, relegated to the status of unactionable digital noise that municipal command centers are neither equipped nor incentivized to monitor, parse, or operationalize.

5.3. The Bureaucratic Paralysis of Legacy Triage Architectures

Conversely, the orthodox institutional response to this digital cacophony has been the deployment of centralized grievance redressal portals and telephonic helplines (e.g., legacy 311 or 1916 systems). However, these legacy architectures are deeply hobbled by byzantine, user-hostile interfaces, opaque operational workflows, and a profound lack of bi-directional transparency. When a citizen submits a data point into these systems, it enters a procedural "black box," immediately severing the feedback loop and fostering a pervasive sense of civic disenfranchisement. More critically, from the perspective of the administrative apparatus, these systems generate an intractable operational bottleneck. Municipal authorities and

command-and-control centers find themselves besieged by an avalanche of unstructured, highly redundant data inputs. Human operators are forced to expend exorbitant cognitive overhead executing manual, heuristic-based triage—reading, interpreting, and re-keying raw textual or auditory transcripts into siloed database architectures.

5.4. The Redundancy Paradox and Spatial-Temporal Inefficiencies

This manual triage paradigm directly engenders the "Redundancy Paradox" of civic administration. Because legacy systems lack semantic spatial awareness and real-time deductive intelligence, a single, highly visible infrastructural failure (such as an overflowing sewage matrix in a high-density sector) will inevitably generate dozens, if not hundreds, of isolated, duplicate complaint vectors. In the absence of an algorithmic clustering mechanism, human operators iteratively process these duplicate vectors as distinct, parallel incidents. This results in the catastrophic misallocation of kinetic resources: multiple, redundant maintenance crews and vehicular assets are simultaneously dispatched to identical geospatial coordinates, hemorrhaging municipal budgets, inflating administrative overhead, and artificially elongating the Service Level Agreement (SLA) response times for other, potentially more critical, infrastructural hazards languishing in the unprocessed queue.

5.5. The Erosion of the Citizen-State Social Contract

The cumulative macroscopic impact of these systemic inefficiencies is the gradual, yet inexorable, erosion of the foundational social contract between the governed and the governing apparatus. When localized priorities are consistently obscured by administrative bloat, and when inter-departmental coordination relies on fragile, asynchronous communication protocols rather than integrated, interoperable intelligence layers, the resulting operational paralysis breeds systemic civic apathy. The lack of a transparent, auditable, and mathematically optimized tracking layer means accountability is diffused, ground-staff performance metrics remain unquantifiable, and the citizenry is perpetually alienated from the very infrastructure they finance.

5.6. The Imperative for an Intelligent, Agentic Synthesis

Therefore, the definitive problem facing modern municipal governance is the absence of a unifying, AI-driven middleware layer capable of synthesizing the frictionless engagement mechanics of a localized social network with the rigorous, policy-compliant execution pipelines of enterprise resource management. There is an urgent, undeniable market imperative to architect a socio-technical solution that can autonomously ingest unstructured, multi-modal civic grievances, apply advanced natural language processing (NLP) and vector-based semantic clustering to mathematically eliminate data redundancy, and deploy assistive, human-in-the-loop agentic workflows to dynamically route, prioritize, and orchestrate ground-level remediation. Until this cognitive gap is bridged, the administration of urban ecosystems will remain perpetually reactive, economically inefficient, and structurally divorced from the lived realities of its citizenry.

CHAPTER 6
OBJECTIVES

The Mudda project aims to establish a user-friendly platform for reporting and addressing civic concern to promote engagement and communication between residents and local governments. This can be accomplished by:

1. **Creating Easy Access to Reporting Problems:**
   Developing a mobile application that enables residents to report concerns including potholes, flooding, damaged street lights, trash, etc., without needing to understand how to file formal complaints.
2. **Increasing Transparency & Accountability:**
   Providing residents with real-time access to the status of their reported concerns, the assigned department to resolve the concern, and any follow-up required will instill confidence in government operations and improve the feeling of responsibility among government agencies.
3. I**ncreasing Efficiency in Government Response:**
   Providing a unified digital platform for government officials to manage, rank, and provide a resolution to reported concerns will allow for improved efficiency. The structured data, location tags and dashboards will enable better tracking of ongoing concerns.
4. **Fostering Community Involvement:**
   Allowing residents to see, comment and endorse (by liking/up-voting) problems reported by other residents will create an opportunity for residents to become more educated about problems within their community and will create greater collective pressure on local governments to address problems.
5. **Issue tracking based on geographic location:**
   Geographic location tracking through geotagging should enable users to precisely identify the geographic location of various issues. By visualizing this information on a map, authorities can use this data to prioritize issues based on the severity of the issue and their location.
6. **Support media reports:**
   Users should be allowed to submit photos with their posts to provide visual confirmation for authorities to improve the quality of the user complaints.
7. **Create a notification system:**
   The notification system is designed to provide citizens with timely updates regarding the status of the raised issue, the progress the government has made on the raised issue and any other recent developments. This is done to create timely communication between citizens and the government.
8. **Resolving Issues using AI Planning:**
   The system integrates an AI-powered planning engine that assists government officials in generating structured, policy-aligned resolution plans for reported civic issues. By analyzing issue details, location data, historical cases, and relevant guidelines, the AI produces a step-by-step action plan to support faster and more consistent decision-making while maintaining human oversight.
9. **Executing Resolution Plans Through Temporal-Orchestrated Tools**

Approved AI-generated plans are converted into executable workflows using a Temporal-based orchestration engine. Each step is mapped to predefined operational activities such as dispatching personnel, sending notifications, or updating records. The orchestration system ensures reliable, auditable, and systematic execution of issue resolution processes.

CHAPTER 7
TOOLS/PLATFORM USED

7.1 Detailed Frontend Architecture

The decision to utilize Flutter for the citizen-facing application was driven by the need for a cross-platform solution that ensures feature parity across Android and iOS. By using a single codebase, the development cycle was significantly accelerated. The UI heavily leverages Flutter's widget-based architecture to construct responsive layouts , ensuring that the hyper-local social components—such as upvoting, commenting, and sharing —render fluidly on various screen sizes. Furthermore, the Next JS framework was selected for the web and mobile-based browser dashboards to provide administrative users with a robust interface for visualizing workflow orchestration and issue analytics.

7.1.1 Feature-First Layered Architecture (Frontend)

To ensure the Mudda mobile application remains scalable, testable, and maintainable, the frontend codebase was structured using a Feature-First, Layered Architecture. Rather than grouping files by type (e.g., placing all UI screens in one folder and all models in another), the codebase is modularized by feature (e.g., Auth, Issues, Voting, Comments). Within each feature module, the architecture is strictly divided into four distinct layers:

- ➔ **Presentation Layer (UI):** Contains the Flutter Widgets and Screens. This layer acts as a "Passive View," strictly listening to state changes from the application layer and rebuilding the UI accordingly without executing raw business logic.
- ➔ **Application Layer (State Management):** Powered by Riverpod Notifiers (e.g., IssueListNotifier). This layer manages the state of the application, handles user interactions, and interacts with repositories to fetch or mutate data.
- ➔ **Domain Layer (Business Logic):** Contains the core business objects and data models (e.g., Issue, User, Comment). These are pure Dart classes, generated using freezed and json_serializable for immutable state and safe JSON parsing.
- ➔ **Data Layer (Infrastructure):** Responsible for external communications. This includes API Services utilizing the DIO HTTP client for network requests, and Local Storage services handling data persistence for offline capabilities.

7.1.2 State Management and Routing

Mudda utilizes **Riverpod** for robust, compile-safe state management. By defining dedicated notifiers for each feature, the application isolates state mutations, making the UI highly reactive and predictable. For navigation, the application implements **GoRouter**, providing declarative routing with built-in authentication guards. This ensures that unauthorized users cannot access protected routes (like the Issue Submission screen) and enables a persistent bottom navigation shell for seamless user experience.

7.2 Deep Dive into AI and Backend Integration

The backbone of Mudda's data processing is built on Spring Boot (Java), chosen for its

highly secure and modularized architecture. This backend acts as the central router, managing role-based access control for citizens versus government officials.
Operating alongside this is the Python Automatic AI Workflow microservice. This is not merely a static classification model, but a smart agentic AI system. When an unstructured text or image payload is submitted by a citizen, this Python service uses Natural Language Processing to extract metadata, assess sentiment, and orchestrate a resolution plan. If hate speech is detected, or if the issue is a duplicate of an existing geospatial cluster, the autonomous workflow flags the issue before it reaches a human official, drastically reducing administrative overhead.

The process of developing Mudda entailed using a contemporary technology stack designed for mobile-centric civic engagement platforms; specifically, this included the following major components:

a. **Front-End Development:**

- ➔ Flutter Framework - Utilized to design a cross-platform mobile application that would present the same user interface on Android as well as iOS devices.
- ➔ Android Studio and Visual Studio Code - Both Integrated Development Environments used to develop the user interface, organize the project's structure, debug, and test the application on emulators.
- ➔ Next JS - Web + Mobile based browser dashboard for admin login and to see issue status, analytics, workflow and human in the loop integration for automatic issue resolution. The dashboard also includes visualization of workflow orchestration.

b. **Back-End Development:**

- ➔ Spring Boot (Java) - Provides the ability to create highly secure, very modularized, and highly efficient REST-Based Application Programming Interfaces for all aspects of managing issues, authenticating users, media handling and role based dashboards for users.
- ➔ Python - Automatic AI Workflow microservice + Model Training and development for issue duplication, hate speech recognition, AQI Assessment, etc. AI Workflow service is a smart agentic AI system with built in workflow orchestration that automatically makes a plan for a said issue and autonomously calls the other microservices/API endpoints to carry out plans accordingly.

c. **Databases:**

- ➔ PostgreSQL (AT Neon) - Acts as the primary data repository, providing a relational database structure for all other entities within the system including users, government roles, issues and tracking.

- ➔ Pinecone Instance - Vector Database for RAG-microservice, to find similar documents for adding further context into AI Planning Engine.

d. **Cloud and Deployment:**

- ➔ Amazon Web Services (AWS) - Provides the capabilities to host the back-end application, scale and make the application highly available and deliver secure APIs to the front-end via AWS.

e. **DevOps and Version Control:**

- ➔ GitHub - Provides the tools needed to manage source code, work collaboratively with others, branch source code, and to perform code reviews.
- ➔ GitHub Actions (CI/CD) - Provides a mechanism for creating automated build and deployment pipelines, thereby enhancing productivity and streamlining software development and releases.

f. **Documentation and Team Coordination**

- ➔ The team uses Notion as the primary tool for documenting project information (e.g. Project Meeting minutes, tasks, Planning resource) because there is no official Project Management Tool available for the project.

CHAPTER 8
DESIGN METHODOLOGY

The Mudda project utilized an Agile Software Development Methodology which focused on ongoing improvement, flexibility, and continuity of development through iterative development cycles.

8.1. Iterative Development
Development occurred in short development cycles (sprints). At each end of each sprint functional components (e.g., logging in, posting issues, viewing dashboards) were completed, tested and assessed.

8.2. Continuous Feedback
Frequent team discussions allowed quick feedback on newly developed features, thereby enabling team members to enhance user interface design elements, streamline back-end processes and optimize API integration through practical testing rather than having to wait until the end of development.

8.3. Concurrent Workstreams
Agile's ability to have the Front End, Back End and DevOps teams work in parallel greatly reduced the overall development time while eliminating bottlenecks.

- Frontend – Developed and tested user interface screens.
- Backend – Built APIs and database schemas.
- AI – Implemented LLM orchestration to align with resolving issues and flagging posts.
- DevOps – Built deployment pipelines and setted up environments.

8.4. Incremental Feature Addition
Key features such as creating a new issue, uploading images, displaying government dashboards, authenticating users, and sending notifications were added sequentially over a period of time rather than being released simultaneously. Each feature was integrated incrementally into the existing system and extensively tested.

8.5. Design Flexibility
Because of iterative development processes, the Agile framework allowed for flexibility as the project progressed. Examples of design flexibility include enhancing the issue tracking process, adding sentiment based prioritization, and streamlining role based access.

CHAPTER 9
CHALLENGES AND ISSUES IDENTIFIED

During the development of the system, several technical and design challenges were encountered. These challenges mainly involved integrating multiple technologies, managing deployment environments, and ensuring that the system performs reliably for users. Identifying and addressing these issues was an important part of the development process.

1. Deployment and Environment Configuration
Deploying the backend application to the cloud environment also presented some challenges. The application was deployed on Amazon Web Services using AWS Elastic Beanstalk and AWS Lightsail, in which we also self-hosted our temporal (orchestration logic) servers. This required proper configuration of environment variables, server settings, and database connections.

Ensuring that the application ran correctly in different environments such as development, staging, testing, and production required careful configuration management. Minor issues related to dependency versions and runtime configurations were also encountered during deployment.

2. Database Connectivity and Management
Another challenge involved configuring and maintaining the database used for storing application data. The system uses Amazon RDS with a PostgreSQL database engine, a temporalio instance self-hosted on AWS, a Pinecone instance as our vector database and an elasticsearch server. Establishing secure connections between the backend server and the database required proper configuration of credentials, network access rules, and database schemas.

During development, issues such as connection timeouts and schema updates needed to be handled carefully to ensure that the data remained consistent and accessible.

3. Map Visualization and Data Representation
Implementing the dashboard that displays reported issues on a map required additional effort. The challenge was to correctly plot issue locations and display meaningful information such as the number of issues reported in a particular area and the types of issues present.

Managing location data and ensuring accurate visualization required proper handling of geographic coordinates and map rendering within the frontend application.

4. Coordination Between Development Teams
The project involved multiple development components including frontend, backend, and DevOps tasks. Coordinating these parts required clear communication among team members to ensure that changes in one part of the system did not affect the functionality of another. Regular updates and version control practices helped manage these dependencies effectively.

There were also challenges associated with introducing Mudda to the market, of which the biggest are :-

5. The Genesis of a Novel Market Ecology

The deployment of Mudda does not merely represent the introduction of a discrete software application into a pre-existing commercial ecosystem; rather, it constitutes the genesis of an entirely novel ontological category within the civic-tech landscape. Traditional paradigms of civic engagement have historically been bifurcated: they either manifest as highly structured, heavily bureaucratic grievance portals (which suffer from chronic user friction and opaque resolution pipelines) or as entirely unstructured, chaotic social media ecosystems (where civic discourse devolves into unactionable cacophony). Mudda fundamentally collapses this false dichotomy. By synthesizing the dopamine-driven, frictionless engagement mechanisms of modern social networks with the rigorous, deterministic workflows of an enterprise-grade AI decision-intelligence layer, we are not capturing market share—we are manifesting a "blue ocean." We are pioneering the "AI-Governance-as-a-Service" (AIGaaS) sector, effectively creating a new market economy where civic sentiment is transmuted directly into automated, trackable infrastructural remediation.

6. The Ontology of AI-Driven Civic Governance

To understand the outcomes of this system, one must first recognize that Mudda engineers a fundamental epistemological shift in how a community interacts with its governing bodies. We are redefining the very nature of a "complaint." In the legacy market, a complaint is a static data point—a form submitted into a void. In the newly minted market catalyzed by Mudda, a complaint is a dynamic, socially validated, AI-processed entity. Our sophisticated NLP and computer vision pipelines autonomously parse, cluster, and route these entities, stripping away the administrative bloat that has historically paralyzed municipal authorities and Homeowner Associations (HOAs). The outcome here is not simply "faster resolution times"; it is the complete infrastructural overhaul of civic accountability. The system enforces an unbroken, symbiotic loop of action, rendering traditional bureaucratic intermediaries entirely obsolete.

7. Formidable Impediments in Market Categorization and Adoption

Forging a new industry standard is intrinsically fraught with structural and cognitive resistance. Because Mudda occupies the liminal space between a social network and a bureaucratic command-and-control system, the primary challenge lies in the cognitive dissonance of the end-users and prospective enterprise clients.

- **The Paradigm Myopia of Procurement:** Government bodies and institutional managers are historically conditioned to procure distinct, siloed solutions—they buy either communication tools or database management systems. Pitching a unified platform that acts as both the public square and the adjudicating authority requires overcoming deeply entrenched procurement mentalities. We must educate the market that an AI co-pilot is not a usurpation of municipal authority, but an augmentation of it.
- **The Friction of "Assistive" vs. "Autonomous" Perceptions:** A significant challenge in this nascent market is navigating the technophobia inherent in public

administration. There is a palpable institutional apprehension regarding the abdication of operational control to artificial intelligence. Mudda must constantly manage the delicate rhetorical balance of promoting its agentic workflows and automated deduplication while assuaging bureaucratic anxiety by strictly framing the AI as "assistive"—a system that suggests, but never executes without human authorization.

8. Overcoming the Cold-Start Conundrum in a Two-Sided Ecosystem

Creating this market necessitates the simultaneous cultivation of two disparate user bases: the citizenry (who demand a seamless, engaging Flutter-based mobile experience) and the administrators (who require a robust, Spring Boot-driven enterprise dashboard). The outcome of Mudda's initial deployment is intrinsically tied to overcoming the classic network-effect paradox. Citizens will not utilize the platform if the authorities are not actively resolving issues through the dashboard, and authorities will not invest in the dashboard if the citizens are not generating a critical mass of structured, actionable data. Thus, the immediate challenge is orchestrating a synchronized, highly localized launch—such as within specific gated townships or targeted municipal zones—to artificially construct a microcosm of this new market, proving the efficacy of the AI-clustering and automated triage in real-time.

9. Anticipated Metamorphic Outcomes

Ultimately, the intended outcome of Mudda is not merely the successful compilation of code or the deployment of scalable microservices, but the sociological reprogramming of civic duty. We anticipate an environment where administrative overhead is reduced by an order of magnitude, duplicate grievances are mathematically eliminated through vector-based semantic clustering, and community trust is algorithmically restored. Mudda is poised to transition civic management from a reactive, crisis-mitigation model into a proactive, predictive, and radically transparent digital utopia.

10. Managing Infinite Scrolling and Memory Leaks

As the number of reported civic issues grows, loading all data simultaneously would crash the mobile client and overload the AWS backend. Implementing an infinite scrolling feed presented a significant challenge. Early iterations suffered from memory bloat due to Flutter retaining off-screen image assets and widgets.

- **Resolution:** The team implemented pagination at the API level (Spring Boot) and integrated Flutter's inbuilt features and some external libraries on the frontend, which lazily loads widgets only when they enter the viewport. Riverpod was utilized to append new data chunks to the existing state list safely, and image caching libraries were integrated to compress and evict off-screen media assets from active memory.

11. Orchestrating the AI Agentic Workflow

Integrating the Python AI microservice was not as simple as making a standard REST API call. Because tasks like Computer Vision analysis and Natural Language Processing (hate speech detection, sentiment scoring) are computationally expensive, synchronous API calls resulted in severe frontend timeouts.

- **Resolution:** The architecture was shifted to an asynchronous, decoupled model. When a user submits an issue, the backend immediately returns a 202 Accepted

status, allowing the user to continue using the app. The payload is then passed to the Python AI workflow in the background. Once the AI completes its deduplication and sentiment analysis, it updates the PostgreSQL database, and the user's interface is updated dynamically on their next refresh.

12. Designing a Deterministic AI Workflow Engine

One of the most technically complex challenges was designing the AI Workflow Engine in a way that balanced flexibility with determinism. Unlike traditional rule-based workflows, AI-driven decision layers are probabilistic in nature. However, government-grade systems require deterministic and auditable outcomes.

The core challenge was ensuring that AI outputs (such as clustering decisions, routing classifications, and prioritization scores) did not introduce unpredictable workflow behavior. Variability in LLM outputs when prompts were not tightly constrained, non-deterministic branching in decision trees based on probabilistic thresholds, difficulty in debugging AI-triggered workflow transitions and maintaining explainability for audit purposes were the problems we encountered while designing an AI Workflow Engine.

Resolution:

- ➔ Introduced structured prompt engineering with strict output schemas that matched our custom compiler to turn it into executable temporalio orchestration steps.
- ➔ Added logging and traceability layers for AI decision explanations.
- ➔ Introduced fallback rule-based workflows when AI confidence scores dropped below acceptable limits.

This ensured that AI remained assistive yet controlled, aligning with public-sector compliance expectations.

13. Temporal Workflow Determinism Constraints

Using Temporal as the orchestration engine introduced a critical architectural constraint: workflows must be deterministic. Since AI inference involves external service calls and non-deterministic computations, directly embedding AI calls inside Temporal workflows initially caused replay failures.

Issues Encountered:

- ➔ Workflow replay errors due to non-deterministic AI API calls.
- ➔ Serialization issues with large AI inference payloads.
- ➔ Handling retries for long-running Computer Vision tasks.
- ➔ Ensuring idempotency during workflow re-execution.

Resolution:

- ➔ Separated AI inference into Activities rather than executing inside workflow code.
- ➔ Used Temporal Signals and Queries to manage state transitions safely.
- ➔ Implemented idempotency keys for AI task submissions.

➔ Externalized heavy payloads into database storage instead of passing them directly through workflow history.

This allowed the orchestration engine to remain stable while still integrating advanced AI capabilities.

14. Long-Running AI Task Management
AI tasks such as image classification, deduplication via vector search, and sentiment analysis can exceed typical HTTP timeout limits. Managing these within a distributed system introduced challenges in reliability and user experience.

Issues Encountered:

➔ Frontend timeouts during synchronous AI calls.
➔ Duplicate AI processing due to retry logic.
➔ Workflow stalls when AI services temporarily fail.
➔ Monitoring long-running tasks across microservices.

Resolution:

➔ Adopted event-driven, asynchronous architecture.
➔ Implemented retry policies with exponential backoff in Temporal.
➔ Added dead-letter queues for failed AI executions.
➔ Built monitoring dashboards to track workflow states and AI processing times.

15. Semantic Clustering at Scale
The AI workflow engine performs semantic deduplication using vector similarity. As issue volume increased, maintaining performance and accuracy became a challenge.

Issues Encountered:

➔ Increased latency in vector similarity searches.
➔ False positives in clustering when thresholds were too low.
➔ Memory strain during batch embedding operations.
➔ Re-indexing challenges when updating embedding models.

Resolution:

➔ Tuned similarity thresholds empirically through test datasets.
➔ Implemented batch processing pipelines for embeddings.
➔ Used asynchronous indexing workflows.
➔ Introduced manual override capabilities for administrators to correct clustering errors.

16. Governance, Auditability, and Human Oversight
Since Mudda operates in civic and administrative environments, AI automation must remain accountable and transparent.

Issues Encountered:

- ➔ Ensuring every AI-triggered action could be audited.
- ➔ Designing workflows where AI suggestions require human approval.
- ➔ Managing override hierarchies in administrative dashboards.

Resolution:

- ➔ Introduced human-in-the-loop checkpoints within Temporal workflows.
- ➔ Logged AI inputs, outputs, and decision rationales.
- ➔ Implemented administrator override mechanisms.
- ➔ Ensured all automated actions were reversible and traceable.

CHAPTER 10
INTERFACE AND DESIGN IMPLEMENTATION

10.1 User Interface Design

The user interface of the system is designed with the goal of making the platform simple and intuitive for citizens to use. Since the application is intended for reporting civic issues, the interface focuses on reducing the number of steps required to submit or track an issue. Clear navigation, readable layouts, and simple visual elements are used so that users can easily understand the functionality of the system.

The interface layout follows a structured design where major features such as reporting an issue, viewing issues on the map, and tracking previously submitted complaints are easily accessible from the main dashboard. Icons and labels are used to guide users through different sections of the application.

Special attention has been given to usability so that even users who are not familiar with complex mobile applications can interact with the platform without difficulty.

10.2 Interface Components of Mobile App

The application interface consists of several main screens that support the different functionalities of the system.

10.2.1 Login and Registration Screen

The login and registration screens allow users to create an account and securely access the platform. During registration, the user provides basic information such as name, email address, phone number, and password. Once registered, users can log into the system and access all features of the application.

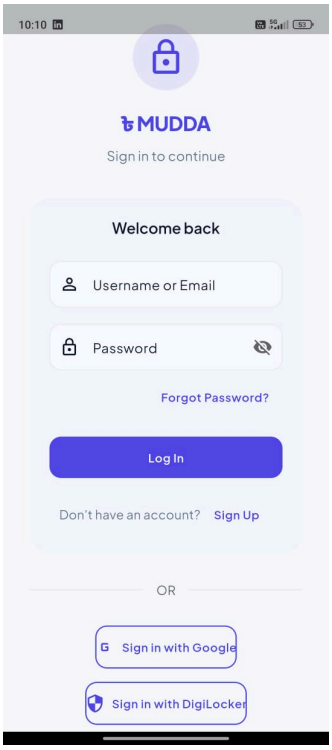


Fig 10.1 Login/Signup Screen

10.2.2 Home Dashboard

The home dashboard acts as the central navigation point of the application. From this screen, users can quickly access the main features such as reporting a new issue, viewing previously submitted issues, and checking updates related to their complaints.

The dashboard is designed to provide quick access to frequently used features so that users can perform tasks without navigating through multiple screens.

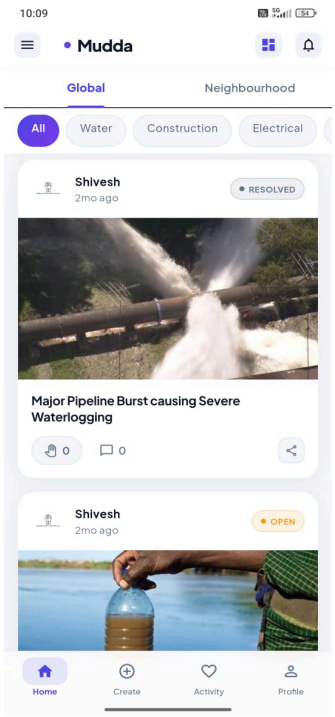


Fig 10.2. Infinite Scroll Social Media-esque Feed

10.2.3 Issue Reporting Screen

The issue reporting screen allows users to submit civic issues that they encounter in their area. Users can provide relevant information such as the type of issue, description, and location details. The interface is designed to guide the user through the reporting process in a simple and structured manner.

This screen ensures that sufficient information is collected so that the reported issue can be properly analyzed and assigned to the appropriate authority. The interface components do not operate in isolation; they are tightly bound to a cloud-native data pipeline. When a user interacts with the Issue Reporting Screen , the media and location metadata captured by the device are transmitted via secure REST APIs to the AWS infrastructure.To handle variable loads, such as a sudden influx of reports in a specific ward, the application is hosted on AWS Elastic Beanstalk. This service dynamically scales the virtual servers provided by Amazon EC2, ensuring that the backend remains responsive. Once processed, the structured data (user details, issue status, and geographic tags) is committed to the Amazon RDS PostgreSQL instance. This structured cloud repository then serves as the single source of truth, instantly pushing updates back to the Flutter frontend so users can see real-time state changes on their Issue Tracking Screen

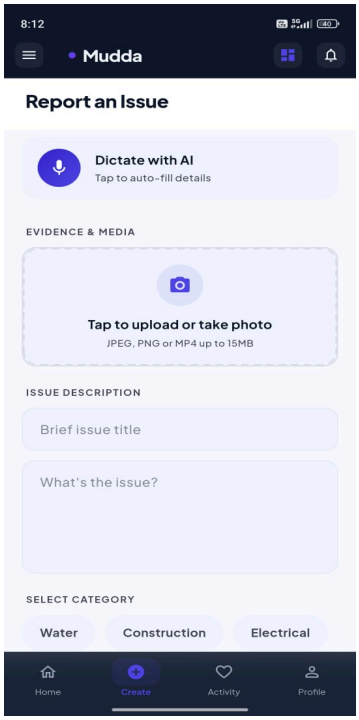


Fig 10.3. Report Infrastructure Issues Screen

10.2.4 Issue Analytics Dashboard

In addition to the standard user screens, the system also provides a dashboard that displays issue-related information on a map. This dashboard visually represents the locations where issues have been reported, allowing users and administrators to identify areas with higher concentrations of problems.

The dashboard also displays useful information such as the total number of issues raised in a particular area and the different types of issues reported. By presenting this information in a visual format, the system helps provide a clearer understanding of civic problems within different regions.

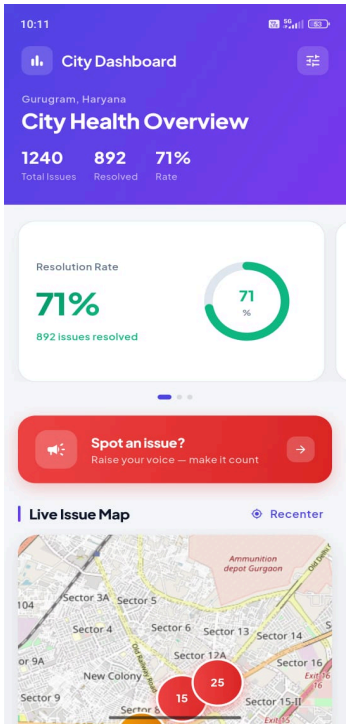


Fig 10.4. Local Dashboard

10.3 Interface Components of Web Dashboard

The Government Web Dashboard is a secure, role-based administrative interface designed specifically for authorized government officials. Unlike the civilian mobile application, which focuses on reporting and tracking issues, the web dashboard is designed for decision-making, workflow orchestration, policy compliance, and operational execution.

The dashboard provides structured visibility into routed civic issues and enables departments to take AI-assisted, accountable action.

10.3.1 Open Issues Management Tab

The Open Issues tab serves as the primary operational interface for government officials. It displays a structured table containing all civic issues that have been routed to the logged-in department based on classification and jurisdiction.

Each issue entry includes:

- ➔ Issue ID and category
- ➔ Location and ward details
- ➔ Citizen-submitted description and media
- ➔ AI-generated clustering status (duplicate or unique)
- ➔ Priority score (based on sentiment and impact analysis)
- ➔ Current workflow status
- ➔ Timestamp and SLA indicators

The table supports filtering, sorting, and searching based on priority, date, ward, and status to help departments manage workload efficiently.

A key feature of this interface is **AI-Assisted Resolution**. Officials have the option to initiate an AI-generated resolution plan based on Mudda's 7-step planning framework. The system analyzes the issue context, relevant policies, historical case precedents, and resource availability before generating a structured action plan.

The AI-generated plan is not executed automatically. Instead, it is presented as a suggested workflow. Upon approval, the official can proceed to the Workflows tab to initiate execution. This ensures that AI remains assistive and human-authorized.

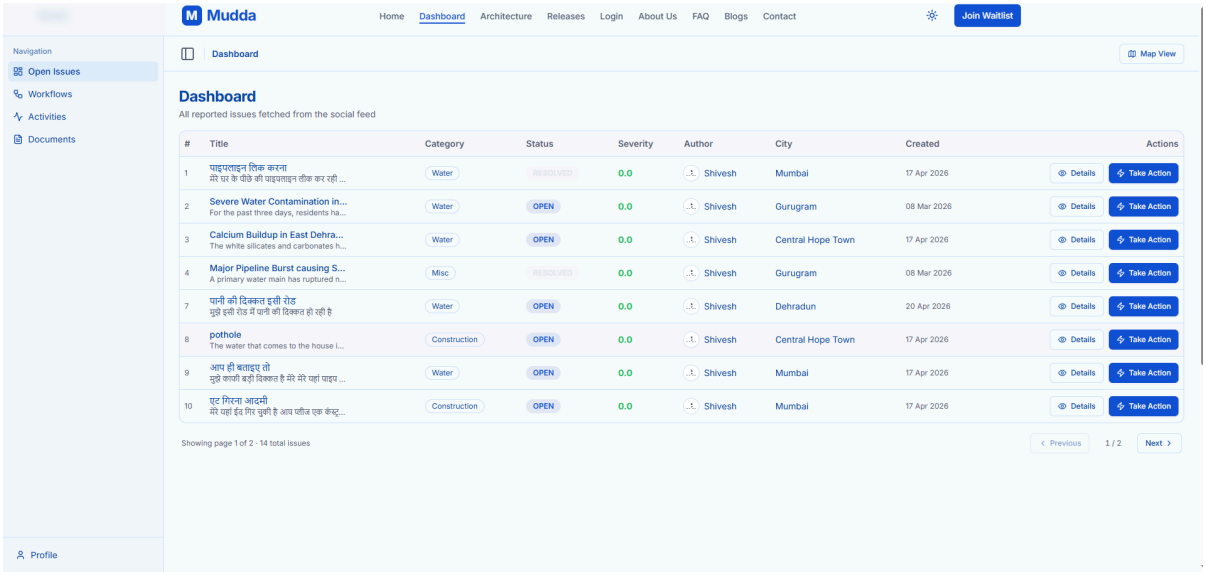


Fig 10.5. Mudda Authority side AI Powered Portal

10.3.2 Workflows Tab

The Workflows tab acts as the orchestration control center for the department. It provides visibility into all workflow instances associated with the department, categorized as:

- ➔ Active Workflows (currently running tasks)
- ➔ Completed Workflows (successfully executed processes)
- ➔ Failed or Escalated Workflows
- ➔ Scheduled or Pending Workflows

Each workflow instance includes:

- ➔ Linked Issue ID
- ➔ AI-generated plan summary
- ➔ Execution steps and timestamps
- ➔ Assigned personnel or vendor details

➔ Current stage in the orchestration lifecycle
➔ Retry or escalation logs

Workflows are executed through a Temporal-based orchestration engine, ensuring deterministic, auditable, and fault-tolerant execution. Officials can monitor progress in real-time and intervene if necessary.

This tab ensures operational transparency and prevents issues from being lost in administrative pipelines.

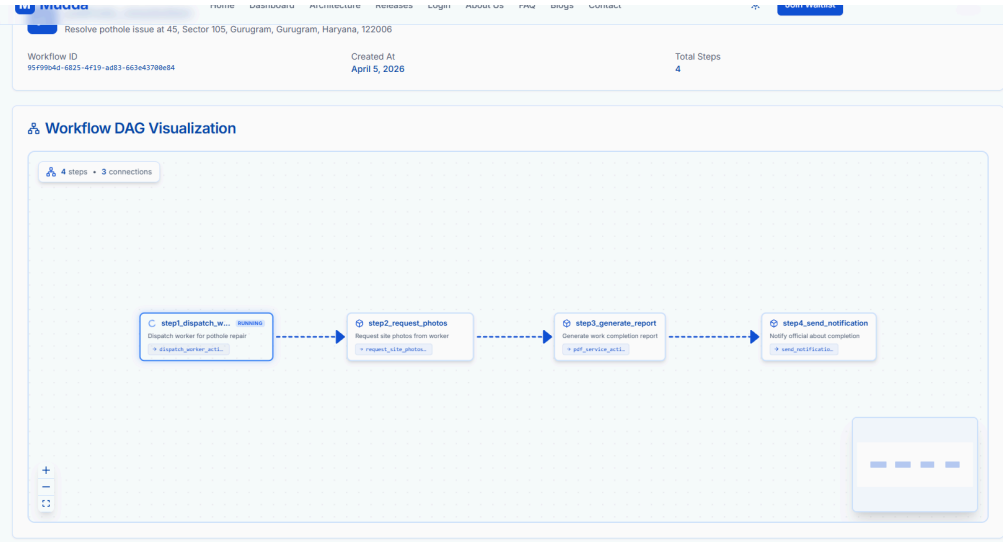


Fig 10.6. AI generated Solution Workflow

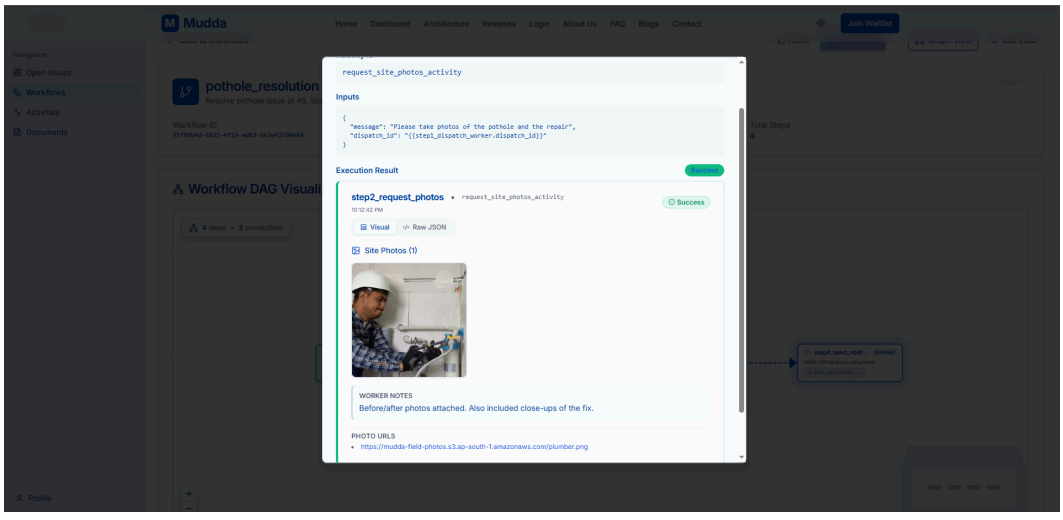


Fig 10.7. Workflow's Individual Activity details after execution

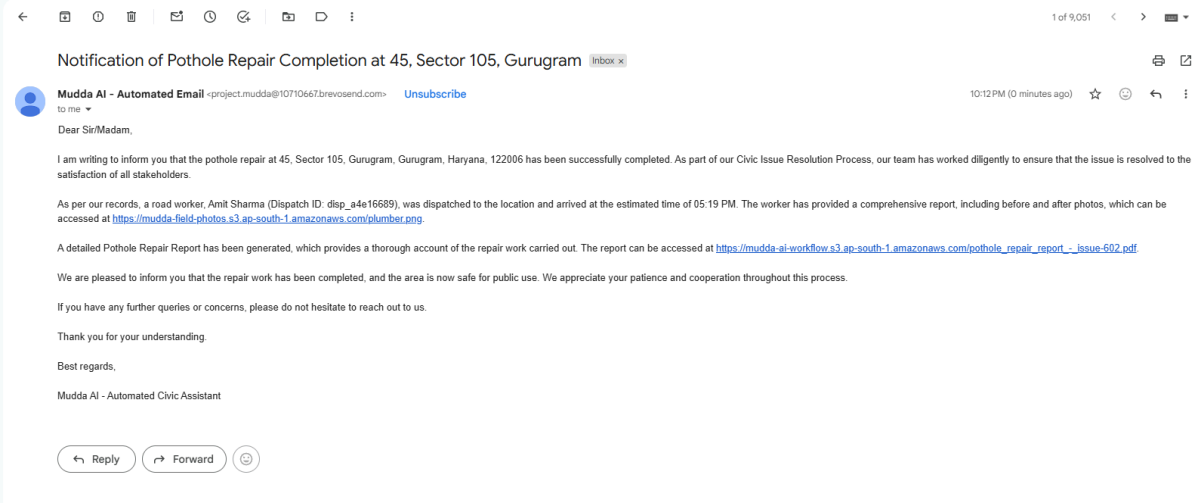


Fig 10.8. Mail and Report of the Issue Resolution

10.3.3 Documents and Knowledge Base Tab

The Documents tab functions as the department's centralized knowledge repository. It provides access to:

- ➔ Government policies
- ➔ Standard Operating Procedures (SOPs)
- ➔ Compliance guidelines
- ➔ Case studies
- ➔ Historical issue resolutions
- ➔ Department-specific operational manuals

This repository serves as the contextual backbone for AI planning. When generating a resolution plan, the AI references these documents to ensure that suggested workflows align with official policies and regulatory frameworks.

Officials can:

- ➔ Search documents by keyword
- ➔ Filter by department or document type
- ➔ Upload updated policy documents
- ➔ View document version history

By integrating knowledge retrieval directly into the dashboard, the system ensures that all resolutions remain policy-compliant and institutionally consistent.

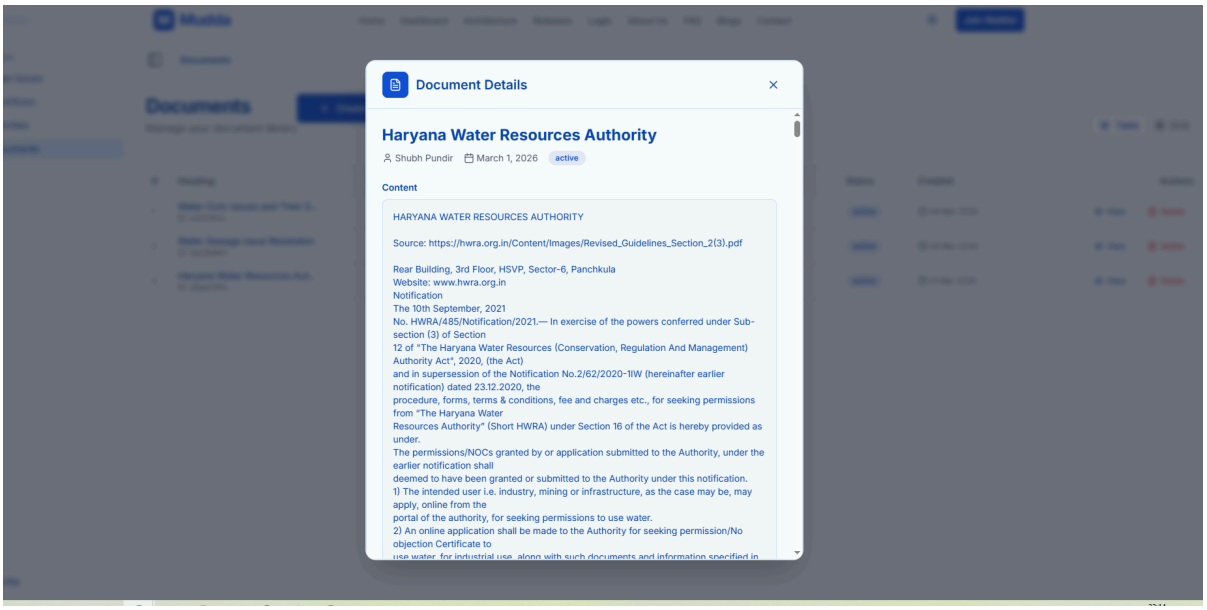


Fig 10.9. Details of document uploaded to and accessible by RAG Service

10.3.4 Activity Marketplace Tab

The Activity Marketplace serves as a deterministic repository for high-fidelity operational primitives. These discrete modules constitute the foundational building blocks that the orchestration engine leverages to synthesize complex, policy-compliant workflows.

Key operational primitives include:

- ➔ Mobilization of field personnel
- ➔ Engagement of external vendors
- ➔ Dissemination of automated communication streams
- ➔ Initiation of on-site verification protocols
- ➔ Documentation of remediation milestones
- ➔ Authorization of fiscal disbursements
- ➔ Finalization and archival of incident lifecycle

Each modular entity is designed for reusability within the Temporal-driven execution layer. Administrative users possess the capability to:

- ➔ Analyze the catalog of available primitives
- ➔ Calibrate operational parameters and SLAs
- ➔ Audit immutable execution telemetry
- ➔ Supervise automated retry and recovery states

This atomized framework empowers municipal departments to institutionalize standardized resolution protocols while facilitating the granular adjustments required for unique civic anomalies.

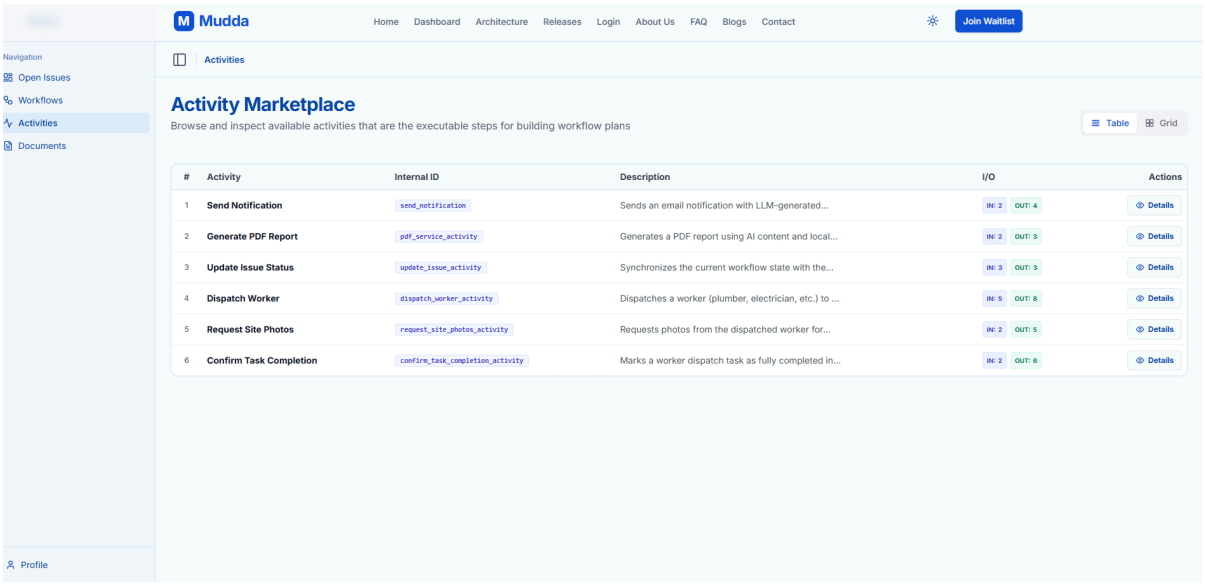


Fig 10.10 List of Activities allowed to the Workflow Planning AI

10.4 Design Implementation

The frontend of the application is implemented using Flutter, which enables the development of a consistent and responsive user interface across multiple devices.

Flutter's widget-based architecture is used to construct the layout of each screen. Various UI components such as buttons, input fields, navigation elements, and lists are implemented using Flutter widgets. These components help maintain a consistent design throughout the application.

Navigation between different screens is handled using Flutter's routing mechanism, which allows smooth transitions between pages. The interface design ensures that the application remains responsive and adapts to different screen sizes.

10.5 Backend Integration

The frontend communicates with the backend server through REST APIs developed using Spring Boot. These APIs handle operations such as user authentication, issue submission, data retrieval, and issue status updates.

Whenever a user performs an action within the application, such as reporting an issue or checking the status of a complaint, the request is sent to the backend server. The server processes the request and returns the appropriate response to the application.

This communication ensures that all user actions are securely processed and the relevant data is stored and retrieved efficiently.

10.6 Cloud Infrastructure Support

To ensure reliable availability and scalability, the backend services are deployed on Amazon Web Services.

The application is hosted using AWS Elastic Beanstalk, which simplifies deployment and automatically manages the environment required to run the backend application. This service handles tasks such as server configuration, load balancing, and application monitoring.

The backend application runs on virtual servers provided through Amazon EC2, which provide the computing resources necessary to process user requests and manage application operations.

10.7 Data Storage

All structured application data is stored using Amazon RDS, specifically with a PostgreSQL database engine.

This database stores information such as:

- ➔ User account details
- ➔ Reported civic issues
- ➔ Issue status updates
- ➔ Location-related information for reported problems

Using a managed database service like Amazon RDS provides benefits such as automatic backups, improved security, and easier database maintenance.

10.8 Continuous Integration and Deployment

The project uses GitHub for source code management and version control. To automate the deployment process, a CI/CD pipeline is implemented using GitHub Actions.

Whenever new code changes are pushed to the repository, the workflow automatically builds the application and deploys it to the cloud environment. This approach helps maintain consistency in deployments and reduces the chances of manual errors during updates.

10.9 Usability and Responsiveness

The application interface is designed to adapt to different device sizes and resolutions. Flutter's responsive layout system ensures that UI elements adjust appropriately across various screen dimensions.

Several usability considerations were taken into account during development, including:

- ➔ Simple and clear navigation structure
- ➔ Minimal steps required to report issues
- ➔ Fast loading of screens and data
- ➔ Easy readability of text and interface elements

These design choices help ensure that the application remains efficient, accessible, and user-friendly.

10.10 Offline Support and Data Synchronization Strategy

A critical requirement for a civic engagement application operating in developing regions is resilience against poor network connectivity. Mudda addresses this through a comprehensive offline-first data caching strategy.

10.11 Local Caching Mechanism

When a user launches the application and loads the Issue Feed, the Repository initiates a network request via the Service. Upon a successful response from the backend, the JSON payload is not only mapped to Dart objects for the UI but is also serialized and stored locally using the Cache Service (via SharedPreferences).

10.12 Offline Fallback Protocol

In the event of network failure or a timeout (simulating a user in a low-connectivity zone), the application catches the network exception and seamlessly falls back to the locally cached data. The Riverpod state notifier updates to a loaded state with an offline flag as true. This triggers the UI to render the previously fetched issues alongside an "Offline Banner," ensuring the user is aware they are viewing cached data while still allowing them to browse civic problems without interruption.

10.13 Optimistic UI Updates for Social Engagement

To maintain a hyper-responsive "social media" feel, hyper-local interactions such as upvoting or commenting utilize optimistic UI updates. When a user upvotes an issue, the UI state is updated instantly, anticipating a successful server response. The API call is processed in the background. If the request fails, the application gracefully reverts the UI state to its previous configuration and alerts the user, ensuring data integrity without sacrificing perceived performance.

CHAPTER 11
OUTCOMES

It is anticipated that as a result of the adoption of the Mudda application, numerous benefits will be reaped from the system:

1. **Efficient Issue Reporting Process:**
   Citizens will no longer require going into the office, utilizing the telephone or going through many bureaucratic steps to report a civic problem because they can do so through their smartphones in the time it takes to check Facebook.
2. **Improved Transparency:**
   Users will have the ability to view the real-time status of their reported issues, along with who is working on them (government authority), and also be able to view the number of similar issues that have been raised in their locality by others, which will help to increase the level of trust in the grievance redressal system.
3. **Faster Issue Resolution:**
   With online access to all problems and issues reported to the government, government officials will have a dashboard that will allow them to view and categorize all issues into one place and quickly assess and resolve public issues.
4. **Enhanced Citizen Engagement:**
   Because the Mudda application has a social media-like format, it will engage more citizens in the posting of issues, liking, commenting on and sharing issues, which will in turn raise public interest and involvement in the local government.
5. **Data-Driven Governance:**
   Government authorities will be able to utilize the trends in data (for example: how many issues have been reported in each region, the type of problem being reported, the frequency of reporting) to make decisions based on evidence and allocate government resources more effectively and to make improvements on a more permanent basis.
6. **Visual Documentation of Problems:**
   The availability of photographic or video evidence for all issues submitted will greatly reduce the uncertainty surrounding the assessment of an issue by the relevant authority through enhanced understanding and reduced mis-interpretation.
7. **Accountability of Authorities Increased:**
   With real time tracking of complaints; public visibility of complaints and status updates; the degree to which authorities are held accountable and are responsible for resolving issues will significantly increase.
8. **Monitoring by Communities:**
   Communities can monitor community issues, upvote issues of greatest concern to them, and see other communities face similar concerns.
9. **Scalable System:**
   Mudda will be developed on a highly scalable application; thus, enabling other future enhancements i.e., automating classification of issues, detecting duplicate instances of issues, implementing multi-lingual capabilities.

In summary, Mudda is committed to changing the existing slow, non-transparent and non-participative system of community reporting of civic-related issues into an efficient, transparent and participative digital reporting service that is community-centric.

CHAPTER 12
GANTT CHART

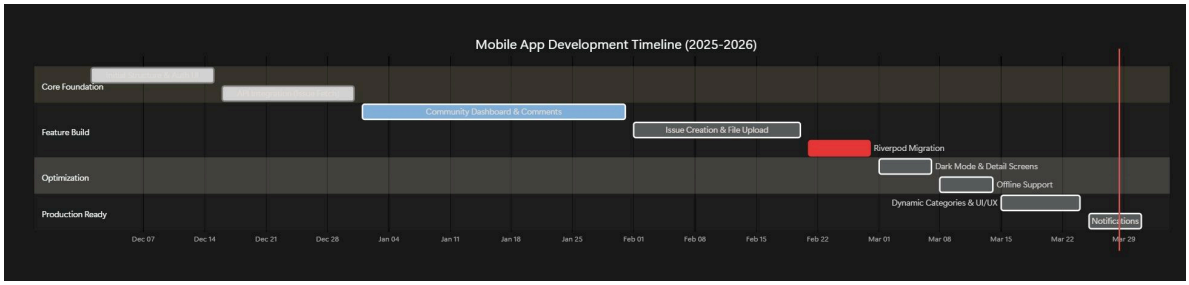


Fig 12.1 Mobile App Development Gantt Chart

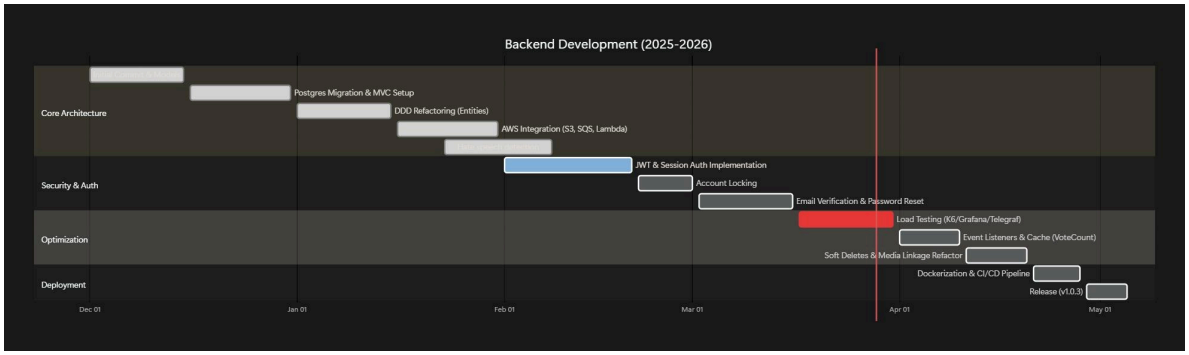


Fig 12.2 Backend Development Gantt Chart

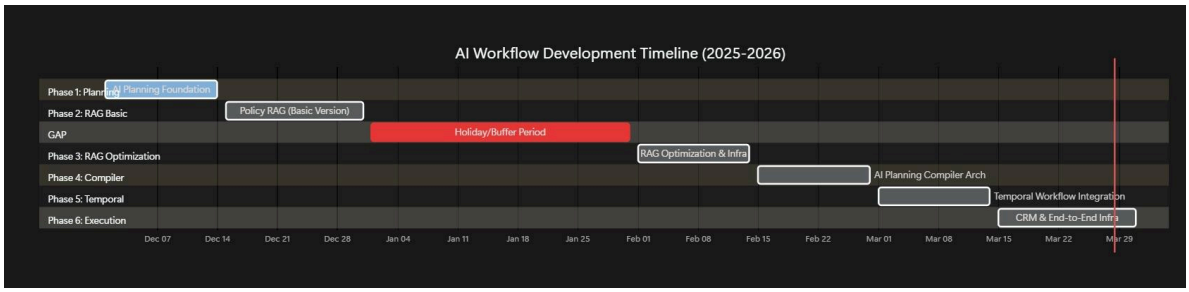


Fig 12.3 AI Workflow Development Gantt Chart

CHAPTER 13
RESPONSIBILITY CHART

Table 13.1 Responsibility chart

Team Member	Role	Responsibilities
Saumil Kulshreshtha	Frontend Lead	• Designing and developing the user interface of the mobile application using Flutter • Ensuring a user-friendly and intuitive experience for issue posting and tracking • Implementing media upload features. • Coordinating with Backend and DevOps teams to integrate frontend with APIs
Shivesh Kumar Dhiman	DevOps/ Frontend	• Assisting in frontend development tasks • Managing application deployment pipelines (CI/CD) • Setting up and maintaining development, testing, and production environments • Configuring and managing cloud infrastructure. • Ensuring proper cloud resource allocation, scalability, and reliability
Shubh Pundir	AI Models/ Backend	• Implementing AI functionalities (e.g., automatic issue categorization, duplicate detection) • Developing backend APIs using Spring Boot for issue management • Integrating machine learning models with the backend • Handling data preprocessing and model inference services
Vikas Kumar	Backend Lead	• Leading the backend development team • Designing and implementing the database schema using PostgreSQL and MongoDB • Creating secure and efficient REST APIs for issue posting, tracking, and user management • Ensuring scalability and performance of the backend system

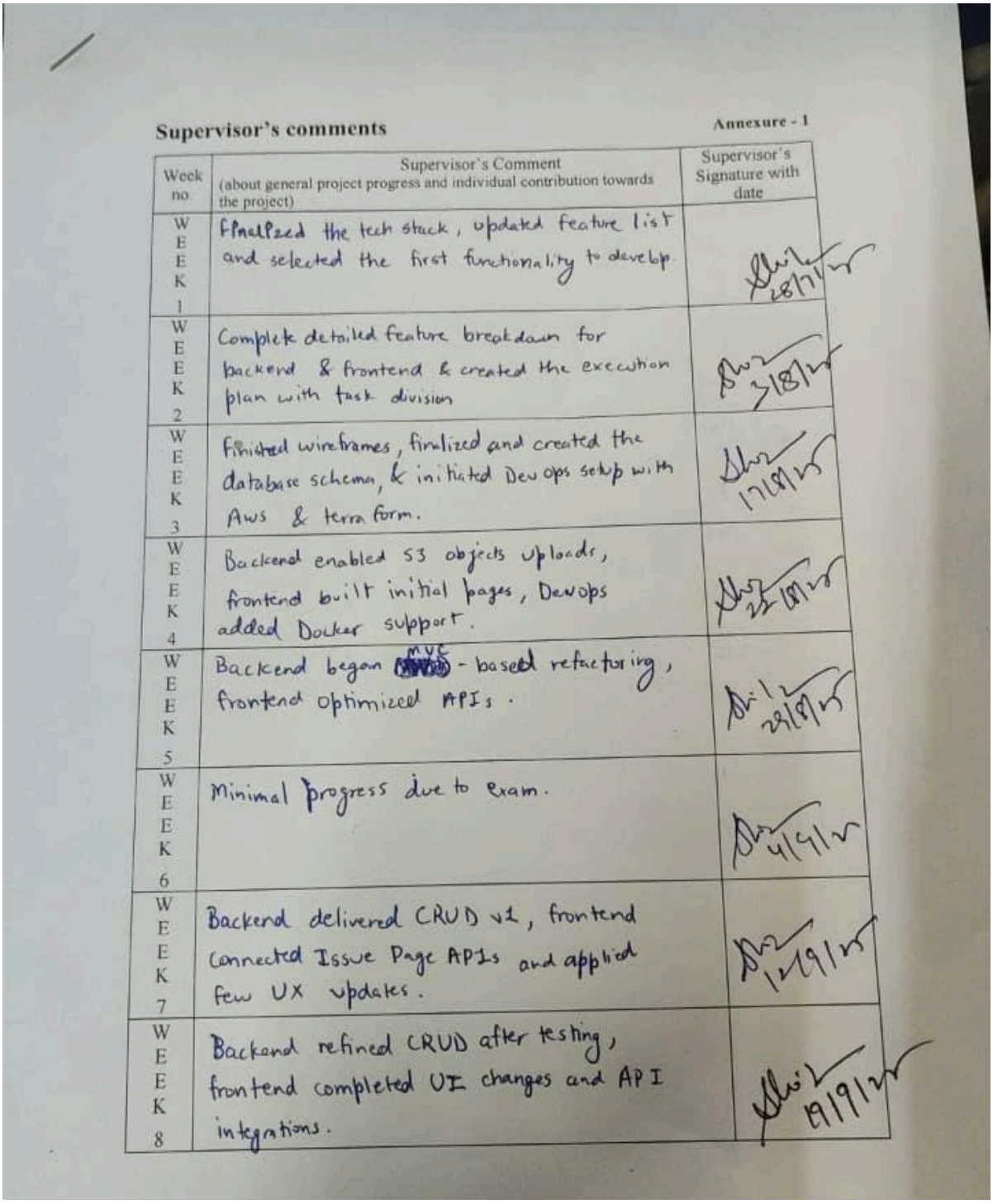
CHAPTER 14
REFERENCES

[1] MoHUA, G. (n.d.). *Welcome to the SWACHHATA platform*. Swachh Bharat. https://www.swachh.city/
[2] Gov Of AP. (n.d.). *Commissioner & director of Municipal Administration government of Andhra pradesh*. PURASEVA | Commissioner and Director of Municipal Administration. https://cdma.ap.gov.in/en/puraseva-3
[3] Greater New Okhla Industrial Development Authority. (n.d.). *Gnida Street light app*. Google. https://play.google.com/store/apps/details?id=gnida.street.gnidastreetlightapp&amp%3Bhl=en_US
[4] Home - bangalore water supply and Sewerage Board. (n.d.). https://bwssb.karnataka.gov.in/english
[5] JobX Robot Pvt. Ltd. (n.d.). *ICRF App*. ICRF. https://www.icrf.org.in/index.html
[6] MCD. (n.d.). MCD 311 App. https://mcdonline.nic.in/portal
[7] MeitY. (n.d.). *Umang App India*. UMANG. https://web.umang.gov.in/
[8] *NetaG app(Unpublished app)*. NetaG. (n.d.). https://www.appbrain.com/app/netag/com.netag

CHAPTER 15
GITHUB AND DRIVE LINKS

1. Civilian Backend repository: https://github.com/Cipher3003/Mudda.git
2. Mobile Flutter app repository: https://github.com/DivSaumil/mudda_frontend.git
3. AI-Workflow repository: https://github.com/ShubhPundir/Mudda-AI-Workflow.git
4. RAG repository: https://github.com/ShubhPundir/Mudda-RAG
5. Web Dashboard repository: https://github.com/ShubhPundir/Mudda-Dashboard
6. Governmental Backend repository: https://github.com/ShubhPundir/Mudda-Government-Backend
7. Hate speech detection model: https://drive.google.com/drive/folders/1y5NoTcsKrmVWkNrMalTdIlpHEr_8Sc80?usp=drive_link
8. Computer vision detection model: https://drive.google.com/drive/folders/1pvOTghK6L6ZJfwsZ4JLQyoMEgiXoZtmg?usp=drive_link

CHAPTER 16
ANNEXURE I



Supervisor's comments Annexure - 1

Week no	Supervisor's Comment (about general project progress and individual contribution towards the project)	Supervisor's Signature with date
WEEK 1	Finalized the tech stack, updated feature list and selected the first functionality to develop	18/7/25
WEEK 2	Complete detailed feature breakdown for backend & frontend & created the execution plan with task division	3/8/25
WEEK 3	Finished wireframes, finalized and created the database schema, & initiated Dev ops setup with Aws & terra form.	17/8/25
WEEK 4	Backend enabled S3 objects uploads, frontend built initial pages, Devops added Docker support.	25/8/25
WEEK 5	Backend began MVC - based refactoring, frontend optimized APIs.	29/8/25
WEEK 6	Minimal progress due to exam.	4/9/25
WEEK 7	Backend delivered CRUD v1, frontend connected Issue Page APIs and applied few UX updates.	12/9/25
WEEK 8	Backend refined CRUD after testing, frontend completed UI changes and API integrations.	19/9/25

WEEK 9	Backend continued DB work, frontend polished UI & fixed performance issues.	26/9/25
WEEK 10	Backend added validation, security checks and seed generator. frontend improved form validation & error handling.	3/10/25
WEEK 11	Backend completed MVC for major modules, frontend finalized static dashboard & analytics.	9/10/25
WEEK 12	Backend did DB optimization, frontend improved responsiveness & accessibility. Devops tested S3 + Docker deployment	17/10/25
WEEK 13	Added User authentication, seed utilities optimized by Backend team. ~~frontend docs~~ Developed a sentiment & abuse detection model.	31/10/25
WEEK 14	Optimized the detection NLP model. fixes in existing Issues APIs, AI agent development in progress.	7/11/25
WEEK 15	Optimized existing APIs, made project homepage & hosted it. further AI agent development. A computer vision detection model development in progress.	14/11/25
WEEK 16	Backend deployed to AWS, Documentation, demo, final report made.	21/11/25
WEEK 17		

WEEK 18	Frontend started Refactoring and built a base RAG AI.	20/1/26
WEEK 19	Backend is doing migration of database. AI RAG system can now retrieve policy and understand them.	24/1/26
WEEK 20	Backend is doing DDD refactoring. AI optimized RAG infrastructure and progressed with compiler architecture planning. Frontend doing optimization.	30/1/26
WEEK 21	Frontend doing bug fixing and building testing suite. Backend doing AWS integration.	6/2/26
WEEK 22	Backend focused on load testing, rate limiting and optimizing caching. Frontend worked on dynamic categories and UX updates. AI initiated temporal workflow.	11/2/26
WEEK 23	Frontend added offline support functionality. Backend finished media handling logic.	17/2/26
WEEK 24	AI ~~finished~~ finalized CRM and end to end workflow logic, The initial Beta release was success fully deployed.	25/2/26
WEEK 25	Backend completed Dockerization & CI/CD pipelines. AI integration was officially marked as complete.	2/3/26
WEEK 26	Full system integration was executed. The team engaged in rigorous QA, bug fixing & end-to-end testing	16/3/26

WEEK 27	Release App version 1.3.0 of Muddu.	25/3/26
WEEK 28	Implemented RWA features in flutter and planned backend integration	1/4/26
WEEK 29	Made token optimized - temporal compat-ible compiler in AI planning microser-vice	8/4/26
WEEK 30	Implemented and tested RWA features in FastAPI.	13/4/26
WEEK 31	Created token usage dashboard and RAG microservice phantom environ-ments	23/4/26
WEEK 32	Evaluated auto ~~sho~~ issue categori-zation pipelines with fine tuned Roberta models	30/4/26
WEEK 33	Refactored web dashboard UI to have a user friendly chat interface.	6/5/26
WEEK 34	Development of terraform scripts for easy deployment and migration	11/5/26
WEEK 35	Tested government faced backend with AI, flutter, Next.JS and spring microservices using Kafka.	18/5/26

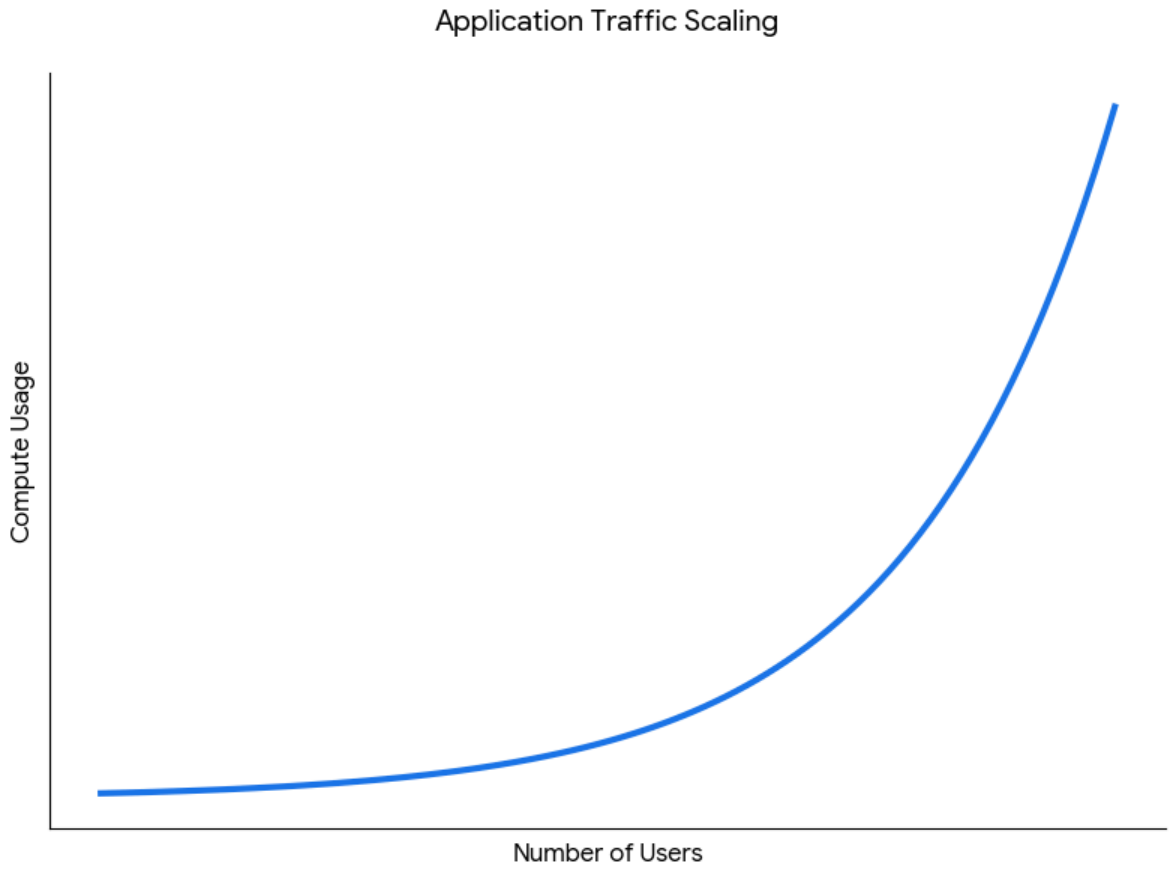


Fig 16.1 Load testing for Application Traffic Scaling

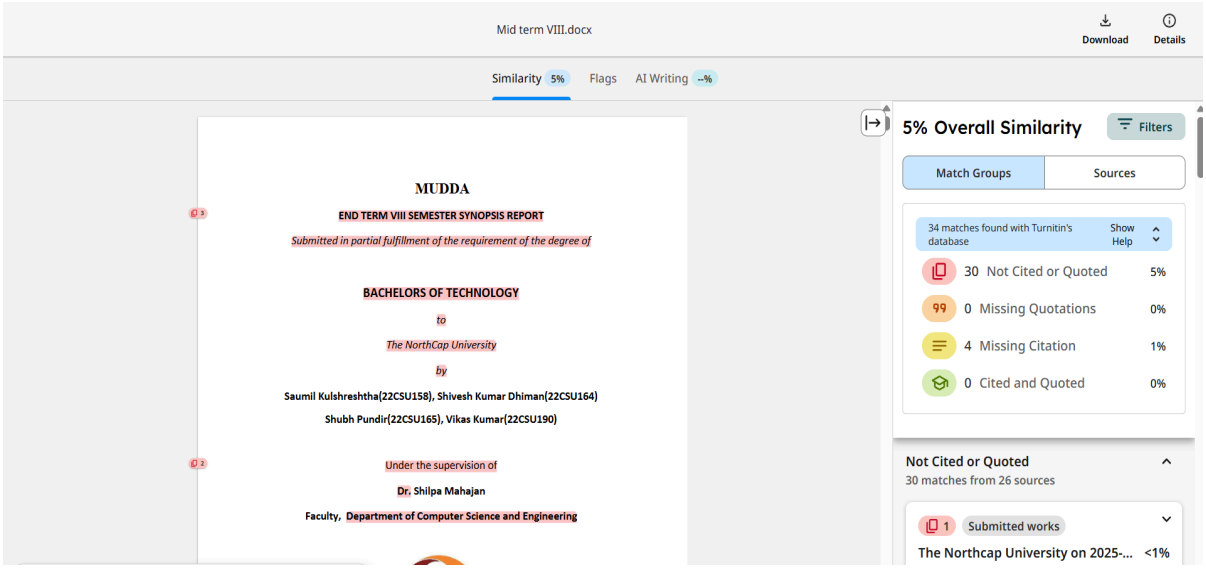


Fig 16.2 Plagiarism and AI detection